Chapter 2

CS1 course package - Tennessee Technological University (TNTECH)

April R. Crockett[1], Gerald C. Gannod[2]

[1]Tennessee Technological University, acrockett@tntech.edu
[2]Tennessee Technological University, jgannod@tntech.edu

Abstract

This chapter presents a Computer Science introductory programming course (CS1) transformation at Tennessee Technological University (TNTECH) in the Department of Computer Science. This chapter also provides access to course materials so that other institutions may be able to adopt and use the materials. Parallel and Distributed Computing (PDC) concepts were introduced into all sections of CS1 starting in Fall 2024 through lecture discussions, unplugged activities during lab class, and (plugged) programming assignments. The primary learning outcomes are for students to (1) recognize terminology associated with parallel and distributed computing topics, (2) identify algorithmic solutions that are representative of parallelism, (3) recognize when a problem solution requires interaction with a remote service, (4) develop programming solutions that are representative of data parallelism, and (5) develop programming solutions that interacts with remote data.

Descriptor Elements

Programming Language Employed: C++

Relevant core courses: CS1

Pre-requisites: no prerequisites

Textbook: zyBooks: Programming in C++ by Frank Vahid and Roman Lysecky [12]

Relevant PDC topics and Blooms level Below is a list of Parallel & Distributed Computing topics that we focused on when designing and creating materials for CS1.

- Data & Task Parallelism (Blooms levels: Understand, Analyze & Apply)
- Distributed Computing via Remote APIs (Blooms levels: Understand, Analyze & Apply)
- Event Handling (Blooms levels: Analyze)

Learning outcomes: The overall learning outcomes added to the CS1 course include the following:

- Recognize parallel computing terms such as data parallelism, task parallelism, task decomposition, speedup, scalability, and pipelining.
- Demonstrate the ability to identify algorithmic solutions that are representative of data parallelism.
- Implement algorithmic solutions that are representative of data parallelism.
- Recognize distributed computing terms such as remote data, distributed data, API, client / server, and JSON.
- Identify when a problem solution requires interaction with a remote service.
- Develop and implement algorithmic solutions for interacting with a remote service.

- Recognize when an algorithmic solution can benefit from the user of multiple threads that communicate via events or similar means of interaction.

Table 2.1 shows the activity-level learning outcomes added to Tennessee Tech's CS1 course with the incorporation of Parallel & Distributed Computing topics. Blooms levels are listed for each module and activity (**RE**membering, **UN**derstanding, **AP**plying, **AN**alyzing, **EV**aluating, **CR**eating).

Table 2.1: TNTECH CS1 Learning Outcomes & Blooms Levels

PDC Concept	**Module Content & Blooms Level**							
	Mod 1 Slides 2.2.2	Unplug. Penny Activity 2.3.1	Mod 7 Slides 2.2.2	Earthqu Program 2.4.1	Mod 8 Slides 2.2.2	Unplug. Flag Activity 2.3.2	OpenMI Lab 2.4.2	Mod 11 Slides 2.2.2
PDC Vocabulary/-Foundations	RE	UN	RE	UN	RE	UN		RE
Data Parallelism		RE			RE	UN	AP,EV	
Race Conditions/Synchronization	UN					UN,AN		
Speedup and Benchmarking							AP	
Distributed Computing			RE	AN,CR				
Event-Driven Programming								RE

Context for use: Tennessee Technological University (TNTECH) is a mid-sized public research university in Cookeville, Tennessee, United States. With an enrollment of about 10,000 students, TNTECH offers more than 40 undergraduate and 20 graduate degree programs across eight academic colleges and schools. The Department of Computer Science at TNTECH offers Bachelors, Masters, and Ph.D. degrees and consists of approximately 700 total students. At present, we have an average of over 200 students each semester enrolled in our CSC 1300: "Introduction to Problem Solving and Computer Programming", which is our (CS1) course. The course is a 4 credit-hour lecture-lab, with 3 hours devoted to lecture and 1 hour devoted to lab. The course is usually split into two sections of lecture and eight to ten sections of lab, where each lab section consists of approximately 25 students each. C++ is the programming language used for instruction, and this course is the first required computer science class in the curriculum of the Computer Science and AI undergraduate degree programs. The CS1

course is taken by students in several additional majors as well, as shown in Table 2.2. Eighty percent of the students in CS1 identify as male and approximately 15% identify as female as shown in Table 2.3 and student race is shown in Table 2.4. Table 2.5 shows the classification of students taking the CS1 course. Students enter CS1 having a wide variety of prior experience in computing and programming. Table 2.6 shows incoming student experience of general computing such as internet searches, email, discord, discussion groups, games, word processing, spreadsheets, organizing files, and organizing folders. Table 2.7 shows incoming programming experience of students entering CS1. This diversity includes exposure to different programming languages, variations in depth, and no programming experience whatsoever. We use the student-selected Experience-Based Grouping (EBG) model to address experience diversity and improve academic success for all students [2–5]. The variation of experience and backgrounds extends to having students from several class standing levels and majors in the course.

Table 2.2: TNTECH CS1 Student Majors by Semester

Major	**Spring 2024**	**Fall 2024**	**Spring 2025**	**Fall 2025**	**Spring 2026**	**Total**
Computer Science	53	153	74	109	32	**421**
Electrical Engineering	36	36	39	62	42	**215**
Mechanical Engineering	19	12	29	21	37	**118**
Computer Engineering	10	10	18	24	18	**80**
Mathematics	4	4	6	10	3	**27**
Physics	1	6	2	10	6	**25**
Music (Live Audio Engineering)	2	2	1	5	1	**11**
Business Information Technology	1	2	0	2	0	5
Civil Engineering	0	1	1	3	0	5
Computer Science Education	1	0	1	1	1	4
Basic Engineering	1	0	0	1	0	2
Business	0	0	1	0	1	2
Interdisciplinary Studies	0	0	0	2	0	2
Manufacturing Eng. Tech.	1	0	0	0	1	2
Nuclear Engineering	0	0	1	1	0	2
Chemical Engineering	0	0	0	0	1	1
Other	0	1	1	6	0	10
Total	**129**	**227**	**174**	**257**	**145**	**932**

Table 2.3: TNTECH CS1 Student Gender Identity

Gender	**Spr 24** n=159	**Fall 24** n=226	**Spr 25** n=168	**Fall 25** n=235	**Spr 26** n=171	**Total** n=959
Male	124	181	128	192	139	764 (79.7%)
Female	26	33	31	33	22	145 (15.1%)
Other	3	6	3	5	2	19 (2.0%)
No Response	6	6	6	5	8	31 (3.2%)

Table 2.4: TNTECH CS1 Race/Ethnicity of Students (Select All that Apply)

Race/Ethnicity	**Spr 24** n=159	**Fall 24** n=226	**Spr 25** n=168	**Fall 25** n=235	**Spr 26** n=171	**Total** n=959
White	120	167	130	171	121	709 (73.9%)
Black/ African American	19	23	17	17	16	92 (9.6%)
Hispanic	7	15	15	24	15	76 (7.9%)
Asian/ Pacific Islander	13	30	8	16	13	80 (8.3%)
American Indian/ Alaska Native	0	5	2	1	1	9 (0.9%)
Middle Eastern/ North African	0	4	2	0	1	7 (0.7%)

2.1 Introduction

Modern computing increasingly relies on parallelism, distributed systems, and data-intensive workflows. National curriculum recommendations such as the ACM/IEEE Computer Science Curricula and the IEEE TCPP Parallel and Distributed Computing (PDC) guidelines, have highlighted the growing relevance of these topics within undergraduate computing education [7, 10]. At Tennessee Technological University (TNTECH), we began infusing PDC concepts into our undergraduate computing curricula staring in Fall 2024 with our CS1 course.

Section 2.2 below describes in detail what was changed in our CS1 course and what specifically was added. We started spending less time on some topics in order to make room for parallelism, distributed computing, and event handling. We also modified some assignments and activities to incorporate PDC topics. Section 11.2.3 highlights the changes and section 2.2.4 discusses challenges faced in updating the curriculum. The details on all new activities are in sections 2.3 and 2.4.

Table 2.5: TNTECH Classification of CS1 Students

Classification	**Spr 24** n=159	**Fall 24** n=226	**Spr 25** n=168	**Fall 25** n=235	**Spr 26** n=171	**Total** n=959
First year (freshman)	100	119	102	125	120	566 (59.0%)
Sophomore	43	78	52	74	39	286 (29.8%)
Junior	13	25	11	30	8	87 (9.1%)
Senior	3	3	3	6	4	19 (2.0%)
Graduate student	0	1	0	0	0	1(0.1%)

Table 2.6: TNTECH CS1 Student Experience with General Computing

General Computing Experience	**Spr 24** n=159	**Fall 24** n=226	**Spr 2025** n=168	**Fall 25** n=235	**Spr 26** n=171	**Total** n=959
Extremely	42	52	37	54	39	224 (23.4%)
Very	63	82	61	76	64	346 (36.1%)
Moderately	44	56	46	58	47	251 (26.2%)
Between Slightly and Moderately	0	16	11	26	10	63 (6.6%)
Slightly	8	10	9	16	5	48 (5.0%)
Very Little	0	8	2	4	4	18 (1.9%)
None At All	2	2	2	1	2	9 (0.9%)

Table 2.7: TNTECH Student Programming Experience Entering CS1

Prior Programming Experience	**Spr 24** n=159	**Fall 24** n=226	**Spr 25** n=168	**Fall 25** n=235	**Spr 26** n=171	**Total** n=959
Extremely	2	3	1	8	0	14 (1.5%)
Very	9	18	12	18	9	66 (6.9%)
Moderately	50	48	37	46	34	215 (22.4%)
Between Slightly and Moderately	0	30	32	26	40	128 (13.3%)
Slightly	68	40	36	43	29	216 (22.5%)
Very Little	0	59	35	61	47	202 (21.1%)
None At All	30	28	15	33	12	118 (12.3%)

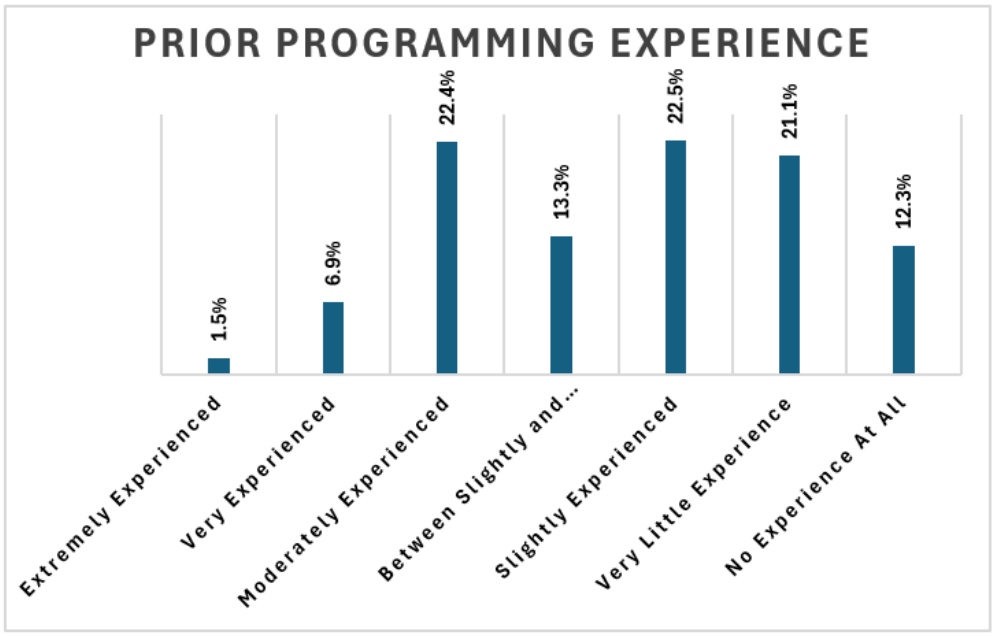


Figure 2.1: TNTECH CS1 Prior Programming Experience

2.2 How CS1 was changed and PDC topics integrated

The CS1 course was revised to introduce parallel and distributed computing (PDC) concepts without fundamentally changing its existing purpose or removing the programming foundations that students need for later courses. Rather than treating PDC as a separate, advanced unit, the course incorporated these ideas into familiar CS1 topics, including computer hardware, loops, arrays, vectors, file processing, and problem decomposition. Approximately four hours of lecture and laboratory time were devoted specifically to PDC content, distributed across the semester so that students encountered the concepts in contexts where they naturally complemented the existing curriculum. This approach allowed students to begin developing a modern model of computation while they were still learning conventional sequential programming.

Some existing topics were condensed to create space in the course. Less instructional time was devoted to detailed coverage of primitive data types, two-dimensional arrays, C-style strings, binary and random-access files, and extensive output formatting. These topics were not necessarily removed entirely; instead, they were treated more selectively because they were considered specialized, less central to contemporary programming practice, or appropriate for reinforcement in later courses. Foundational topics such as variables, selection structures, repetition, functions, arrays, vectors, classes, and file input remained central to the course. The goal was therefore not to replace established CS1 content with PDC, but to periodically reevaluate the relative emphasis placed on different topics and make room for concepts that better reflect modern computing environments. Specifically, a programming assignment formerly centered on arrays and vectors was revised so that students accessed live remote earthquake data, while a lab that had previously focused only on arrays was updated to include OpenMP, and other lab classes incorporated unplugged activities on parallel

programming concepts that are described in Section 2.3.

2.2.1 Integrated Syllabi (Pre and Post)

The course calendar from the syllabus for CS1 at TNTECH as shown in Table 2.8 spans 16 weeks across eleven content modules, covering the standard CS1 arc from introductory programming and C++ fundamentals through branching, loops, functions, arrays, pointers, C++ structs, and classes. The 16th week is final exams week. The PDC interventions, highlighted in the course calendar, are threaded into this existing structure at three points, each chosen because the underlying CS1 content provides a natural and mutually reinforcing context for the PDC concept being introduced.

Table 2.8: TNTECH CS1 Spring 2026 Course Syllabus Calender with Highlighted PDC Content

Module	Week	Assignments	PDC Content Added	Quiz
Mod 1: Introduction	Weeks 1 & 2	Syllabus, Informed Consent Form, Pre-Course Experience Survey, zyBook Chapter 1, Group Selection Form	Informed Consent Form, Pre-Course Experience Survey, zyBook Section over Parallel Programming, Slides (Mod 1 PDC Content in Section 2.2.2)	Mod 1 & 2 Quiz
Mod 2: Prog. Basics	Weeks 2 & 3	zyBooks Chapter 2, Get Computer Set Up, Lab 1 Assignment	Unplugged Penny Search Activity (Mod 2 PDC Content in Section 2.2.2)	Mod 1 & 2 Quiz
Mod 3: Data Types & Math Expressions	Weeks 3 & 4	zyBooks Chapter 3, Lab 2 Assignment, Program 1 Assignment	N/A	Mod 3 Quiz
Mod 4: Branching	Week 5	zyBook Chapter 4, Lab 3 Assignment	N/A	Mod 4 Quiz
Mod 5: Loops	Week 6	zyBook Chapter 5, Program 2 Assignment, Lab 4 Assignment	N/A	Mod 5 Quiz
Mod 6: Files	Week 7	zyBook Chapter 6, Lab 5 Assignment	N/A	Mod 6 Quiz
Mod 7: Functions, Distributed Computing	Week 7 & 8	zyBook Chapter 7, Lab 6 Assignment, Lab 7 Assignment, Program 3 Assignment	Slides, Plugged-In Earthquake Tracker Assignment (Mod 7 PDC Content in Section 2.2.2)	Mod 7 Quiz
Mod 8: Arrays, & Parallel Prog.	Week 9 & 10	zyBook Chapter 8, Lab 8 Assignment, Lab 9-OpenMP Data Parallelism Lab Assignment, Program 4 Assignment	Slides, Plugged-In OpenMP Data Parallelism Assignment, Unplugged Flag Maker Activity (Mod 8 PDC Content 2.2.2)	Mod 8 Quiz

Mod 9: Pointers	Week 11 & 12	zyBook Chapter 9, Lab 10 Assignment, Program 5 Assignment	N/A	Mod 9 Quiz
Mod 10: ADTs & Structures	Week 13 & 14	zyBook Chapter 10, Lab 11 Assignment	N/A	Mod 10 Quiz
Mod 11: Classes & Objects, Event Handling / GUIs	Week 14 & 15	zyBook Chapter 11	Slides (Mod 11 PDC Content in Section 2.2.2) & GUIs	N/A

2.2.2 PDC Topics Integrated

Module 1 New PDC Content

In our CS1 class, computer hardware, the power wall, multiple cores, and introducing parallel programming was discussed during week 1 of lecture. We also assigned the informed consent form and the pre-course experience survey during week 1 of the course. We use a zyBook section created by Casper College 7.3.1 over Parallel Computing and ask a few questions as part of the zyBook chapter homework. Here is a list of PDC material that was used in Module 1:

- PRESENTATION SLIDES_Module 1 Introduction.pptx
- CS1 & CS2 Spring 2026 Informed Consent Form
- CS1 Spring 2026 Pre-Course Experience Survey
- zyBook Chapter 1.5 Parallel Computing

Module 2 New PDC Content

Students go to lab class for 1 hour and 15 minutes once a week starting in week 2. Each lab section is taught by undergraduate teaching assistants. During their first lab, the TAs cover lab policies and then have the students participate in the **Unplugged Penny Search Activity**, discussed further in **Section 2.3**, which helps students get to know each other since it is a group activity and further introduces students to parallel programming concepts.

Module 7 New PDC Content

After covering functions in lecture, we introduce students to distributed computing concepts including:

- getting data from a remote server,

- JSON format and Niels Lohmann's JSON for Modern C++ library [9],
- cURL library [11],
- Make utility and Makefiles [8]

We typically cover the topics above in one 55-minute lecture. Then, students are assigned the **Earthquake Tracker Programming Assignment**. Here is a list of PDC material that was used in Module 7:

- PRESENTATION SLIDES_Module 7 Functions & Distributed Computing.pptx
- PRESENTATION SLIDES_Makefiles_CSC 1300.pptx
- Plugged-In Earthquake Tracker Assignment(Section 2.4.1)

Module 8 New PDC Content

Module 8 begins with arrays and the STL Vector class. Then, we introduce parallel programming and discuss the differences between task and data parallelism. This leads into a lecture introducing students to the OpenMP API [1, 6] for parallel programming in C++. During the following lab class, students participate in an **Unplugged Flag Maker Activity** and then complete the **Plugged-In OpenMP Data Parallelism Assignment** for students to practice what they have learned. Here is a list of PDC material that was used in Module 8:

- PRESENTATION SLIDES_Module 8 Purple_Arrays Vectors Data Parallelism.pptx
- PRESENTATION SLIDES_OpenMP Introduction.pptx
- Parallel Programming & Chrono Program Examples
- Unplugged Flag Maker Activity(Section 2.3.2)
- Plugged-In OpenMP Data Parallelism Assignment (Section 2.4.2)

Module 11 New PDC Content

Module 11 introduces students to Classes & Objects as well as Event Handling & Graphical User Interfaces (GUIs). This module covers ideas and terminology such as event, response, synchronous vs. asynchronous event handling, and GUIs. During lecture, the instructor also demonstrates UNL's Asynchronicity webpage. The Event Handling part of lecture usually takes about 30 minutes to cover. Here is a list of PDC material that was used in Module 11:

- PRESENTATION SLIDES_Module 11_OOP & Event Handling.pptx
- UNL Asynchronicity Demonstration

2.2.3 Highlights

Much of the existing CS1 course remained unchanged following the integration of PDC content. The course continued to emphasize the foundational programming concepts needed for success in subsequent courses, including variables, selection structures, loops, functions, arrays, vectors, pointers, structures, classes, and file input. The overall sequence of the course modules also remained largely the same. Rather than adding a separate PDC unit, the new material was distributed across the semester and connected to topics already being taught.

The most significant changes involved adjusting the amount of time devoted to selected existing topics and modifying several instructional activities. Coverage of primitive data types, extensive output formatting, two-dimensional arrays, C-style strings, and specialized file-processing techniques was condensed to create approximately four hours of lecture and laboratory time for PDC-related instruction. A traditional arrays-and-vectors programming assignment was replaced with the Earthquake Tracker Programming Assignment, which introduced remote data retrieval and distributed computing concepts. An existing arrays laboratory was expanded into the OpenMP Data Parallelism Lab, and unplugged activities were added to illustrate parallel search, workload division, and the effects of using multiple processors.

A key strength of the redesign was that each PDC concept was introduced where it naturally complemented the existing curriculum. Parallel computing was initially connected to computer hardware and multicore processors, distributed computing was introduced alongside functions and file processing, data parallelism and OpenMP were paired with arrays and vectors, and event handling was presented with classes, objects, and graphical user interfaces. This organization allowed students to encounter modern computing concepts without sacrificing the course's primary emphasis on introductory programming fundamentals. It also made the interventions modular, allowing instructors to adopt individual activities or assignments without completely restructuring the course.

2.2.4 Challenges

Preparing the Teaching Team for New Territory

A challenge of teaching the new material was that the material was not only new to the students in the course, but also to the team of teaching assistants. This meant that we would first need to find time to teach the concepts, activities, and lab assignments to the teaching assistants before teaching the students. Finding the time to do this was definitely challenging. There were also some teaching assistants that didn't understand why these concepts were being taught in CS1 and thought it was too much to ask CS1 students to learn about and practice these concepts. We originally didn't expect this confusion and didn't explain the reasoning. However, in the 2nd semester of implementing the PDC concepts we knew that explaining the reasoning must be part of the TA training.

2.3 New Unplugged Activities

During the first week of lecture, students are introduced to programming concepts and computer hardware. As part of the discussion of the central processing unit, we examine the historical Von Neumann architecture, which relies on a single CPU, and the efforts made to improve computational efficiency and speed by increasing the number of transistors. We then discuss how these improvements eventually encountered the power wall, motivating the shift toward multicore architectures. Finally, students learn how multicore processors enable parallel computing, in contrast to traditional serial computing. To solidify this idea, we introduced the **Unplugged Penny Search Activity** in lab.

2.3.1 Unplugged Penny Search Activity

Our first lab class in week 2 (out of 16 weeks) begins with the Unplugged Penny Search Activity, which is introduced in section 1.3.2. Figure 2.2 shows how we store the pennies and distribute during the activity. This is a great unplugged activity for the first lab because it allows students to get to know each other and it introduces the concept of parallelism.

Figure 2.2: TNTECH Organization of Pennies

At TNTECH, we show presentation slides during each phase of the activity and we added an additional phase that is not described in section 1.3.2.

Phase 1 is shown in Figure 2.3. After all groups of students complete Phase 1, the instructors describe that phase 1 represents a purely sequential algorithm where there is only one thread (or worker) performing the task. They explain that there is no parallelism, and the task is completed by single worker from start to finish. The term "sigle-core processing" is introduced. We then ask students, "If we were to divide the pennies up in the beginning and have multiple sorters, do you think this would take more or less time?" and then we proceed to Phase 2.

Phase 2 is shown in Figure 2.4. After all groups of students complete Phase 2, the instructors describe that phase 2 represents two independent tasks (sorting two separate bags) are executed concurrently. We further explain that this represents task parallelism

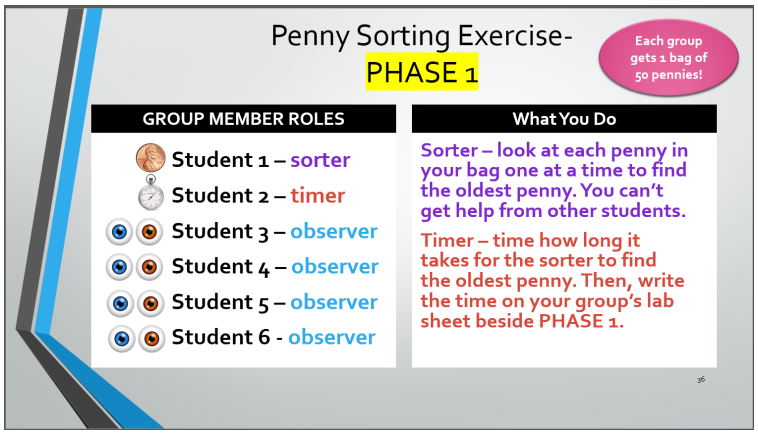


Figure 2.3: TNTECH CS1 Penny Activity Phase 1

where multiple workers perform tasks in parallel and then synchronize (compare results) at the end. The synchronization step is the comparison of the oldest pennies from the two bags, which ensures that the final result is the overall oldest penny. Students are then informally asked the following questions:

1. How did dividing the task between two people (parallel sorting) affect the overall time compared to Phase 1 (sequential sorting)? Did you experience speedup?

2. If each person worked at different speeds, how might that affect the overall time it takes to find the oldest penny?

3. Would adding more people (more sorters with smaller bags of pennies) make this task faster? Why or why not?

The second question typically leads to a discussion about load balancing and how the performance of parallel tasks can be uneven if processors work at different speeds. The third question can encourage students to think about scalability in parallel programs and the potential diminishing returns when adding more workers.

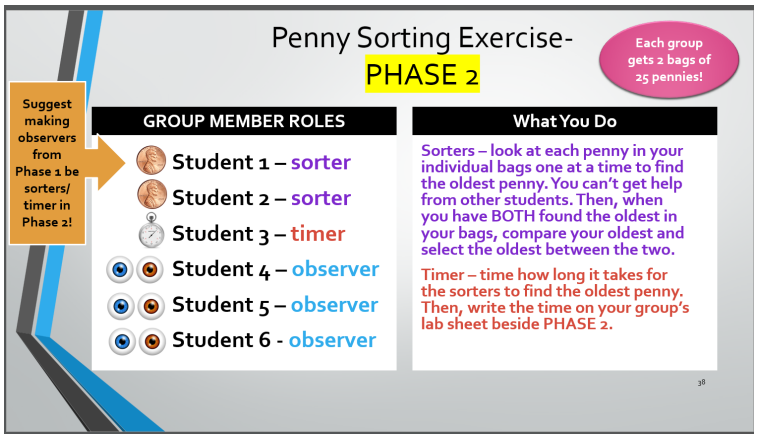


Figure 2.4: TNTECH CS1 Penny Activity Phase 2

Phase 3 is shown in Figure 2.5. After all groups have completed Phase 3, the instructors talk about how this phase demonstrates data parallelism, where the dataset (pennies) is divided among multiple workers. We point out, however, that there is load imbalance, as one student has significantly more pennies to sort than the others. This imbalance reflects real-world scenarios where tasks are unevenly distributed among parallel workers, which can lead to inefficiencies. After finding the oldest penny, the students must communicate (synchronize) to combine their results and identify the oldest penny.

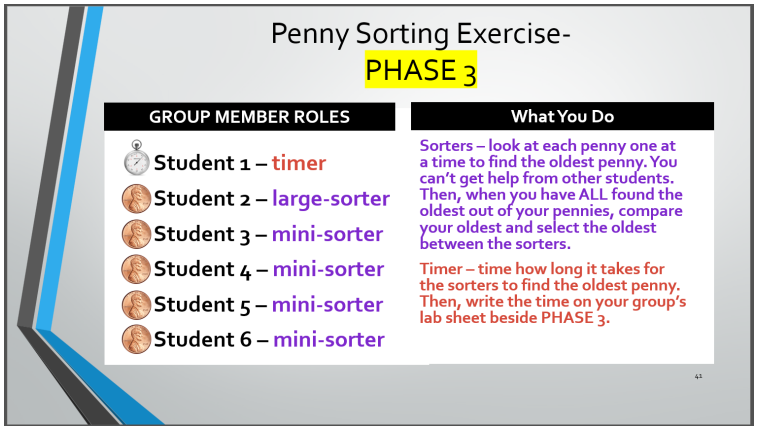


Figure 2.5: TNTECH CS1 Penny Activity Phase 3

Phase 4 is shown in Figure 2.6. According to the lab instructors, students typically have the most fun with this phase. After the students have found the oldest penny in the room, the instructors talk about how this phase represents data parallelism as well, where the dataset (pennies) is divided among multiple workers (each worker only has one penny). We point out that ~~W~~with the data being spread across so many workers, the synchronization step where results need to be merged ends up taking more time than the actual work, and this affects speed-up.

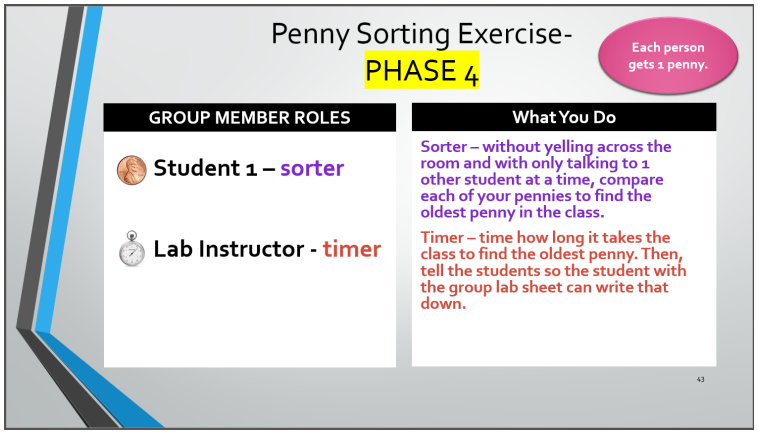


Figure 2.6: TNTECH CS1 Penny Activity Phase 4

Instructional Material (Lectures, Assignments)

Instructor materials for TNTECH's Unplugged Penny Search Activity can be found at `https://github.com/CDER-Center/CS1-CS2_Exemplar_Ebook/tree/main/CS1/chapter2-TTU/Unplugged_Activities/Unplugged%20Penny%20Search%20Activity`.

Challenges

We Accidentally Looked Like Coin Criminals. One of the unplugged activities requires many pennies (if you have a substantial number of students) and so going to the bank and getting 50 dollars of pennies was a much bigger challenge than we thought it would be. First, the bank workers were very suspicious why we would make such a request. The bank teller behind the counter asked me why we needed that many pennies and told us that we would need to wait 15 to 20 minutes because they would have to be retrieved from the vault. Then, a bank manager came out and asked us the same question. While in the waiting room we felt we were being watched very closely. Then, they brought out a box that was small, but it weighed approximately 30 pounds (13.6 kilograms) and so it was much, much heavier than we anticipated. Figure 2.7 SS: shows the heavy box of pennies that led to a heavy workout by loading it on ~~Getting the pennies to~~ my vehicle and then to the instructor's third floor office ~~was more of a workout than originally anticipated~~.

Figure 2.7: TNTECH CS1 Box of 5,000 Pennies

Activity Could be Time Consuming. Because this is the first lab session of the semester, students are introduced to their lab instructors, view an "Introduction to Lab" presentation, complete the in-lab Penny Search Unplugged Activity, and verify that their computers are properly configured to write, compile, and run C++ programs. The unplugged activity requires approximately 30–40 minutes of the 75-minute lab period. As a result, teaching assistants have reported that some students are unable to complete the computer setup process during class. To address this issue, setup instructions are distributed approximately one week before the semester begins, and multiple office-hour sessions are offered throughout the week to assist students who encounter difficulties. Also, some of the

lab instructors combined Phase 1 & 2 into a single phase where they made it a competition and half of the groups would be doing Phase 1 whereas the other half would be doing Phase 2 at the same time. This helped reduce the time it took to complete this activity.

Data and Analysis

Figure 2.8 shows the results to two statements that we asked students to rate their level of agreement given a 7-point Likert scale. These responses refer to the whole lab class including being introduced to lab & lab instructors, doing the unplugged penny activity, and getting their computer set up for the class. Students were also asked, "What is something that you learned from this assignment?" and while the majority of responses were related to getting their computer set up or compiling & running their first program, there were six students who wrote "parallel computing."

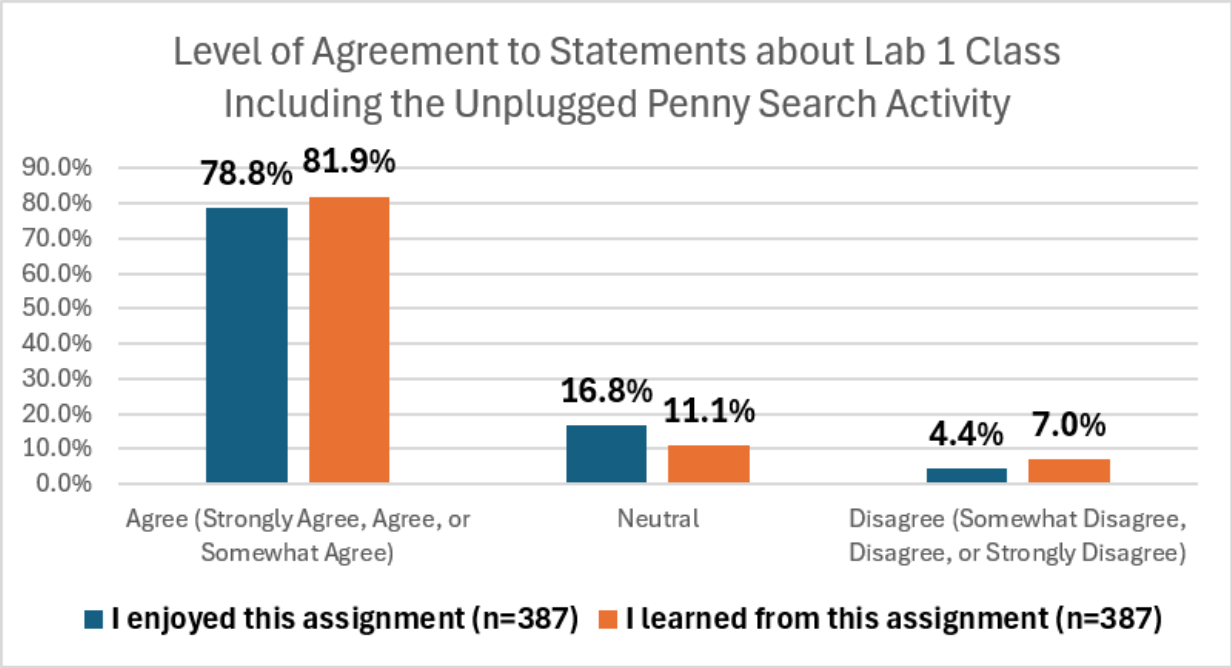


Figure 2.8: TNTECH CS1 Level of Agreement Unplugged Penny Search Activity

2.3.2 Unplugged Flag Maker Activity

Our data parallelism lab in week 11 (out of 16 weeks) begins with the Unplugged Flag Maker Activity, which is introduced in Section 1.3.1 and then students work on a plugged activity detailed in sSection 2.4. The Flag Maker Activity has four scenarios, but instructors can choose to do as many of the scenarios as time permits.

An image of all four scenarios is in Figure 1.3.

Students begin with **Scenario 1**. After the group is done with Scenario 1, they are instructed to write down the time it took to complete. In the first semester of doing this activity, we had students write their times on a lab sheet where each group had a single lab

sheet each. However, starting with the second semester, we started having students write their times up on a white board so that all students could see the times and could compare. This made the activity feel more like a competition or game. We also started handing out small prizes to winning (fastest) groups (shown in Figure 2.9) such as mini resin animals (approx. $10 for 86), flinging chickens (approx. $10 for 42), flinging ninjas (approx. $12 for 40), candy, mini feet (approx. $18 for 40), small squishy toys (approx. $22 for 130), fruit rollerball pens (approx. $9 for 12) and mini hands (approx. $10 for 30), which were purchased from Amazon.com. The mini hands, candy, and mini resin ducks were student's favorite prizes. We found that having students compete keeps all students engaged in the activity. The instructors then talk about how Scenario 1 represents a single processor where every action is sequential.

Figure 2.9: TNTECH CS1 Prizes Used for Flag Maker Activity

Then, students start **Scenario 2**. After recording times and distributing prizes for Scenario 2, the lab instructors ask the students to compare their times for Scenario 1 versus Scenario 2. We then explain that adding a second processor allows for parallel processing, so two stripes of the flag can be worked on simultaneously, cutting down completion time. We also discuss that this demonstrates how distributing tasks between processors can increase efficiency and speedup. We define speedup as the ratio of the time taken to solve a problem on a single processor to the time taken on a parallel system.

Next, students perform **Scenario 3**. During the discussion part of Scenario 3, lab instructors are able to point out that adding additional processors speeds up the work even more and they introduce the importance of synchronization between the processors. We talk about how if one processor finishes early, it sits idle, and how that is an example of underutilized resources. This allows the instructors to talk about how in real systems, load balancing is essential to ensure each processor has roughly the same amount of work to prevent idle time.

Last, students complete **Scenario 4**. During the discussion part of Scenario 4, lab instructors ask the following questions:

1. Was Scenario 4 faster or slower than Scenario 3?

2. Did each person (processor) use a marker, and then pass it to the next person to use – over and over until the work was done?

3. Was each person (processor) sometimes waiting for a marker?

The second question allows us to introduce pipelining as the technique of overlapping the execution of multiple instructions to improve performance. The third question allows us to talk about how when multiple processors are waiting on (competing for) shared resources, this is contention. We also talk about how students hands run into each other during this Scenario and how that represents race conditions - where processors might risk stepping into each other's workspaces at the boundaries.

Instructional Material (Lectures, Assignments)

Instructor materials for the Unplugged Flag Maker Activity can be found here.

Experience and Teaching Tips

The CS1 instructors meet weekly with the CS1 lab instructors (undergraduate teaching assistants) to perform a retrospective of the previous week's lab. We discuss what went well, what could be improved, and if there are any questions to be answered or problems to be resolved. In the Flag Maker retrospective meeting, the lab instructors said that in most labs, students enjoyed the activity except for in labs with smaller numbers of students. For example, a 20 to 30-person lab class enjoyed the activity more than the 10 to 15-person lab class. The TAs felt this was due to lack of being able to make it as much of a competition with fewer students. One TA mentioned that as long as the instructor makes it fun and engaging, students will feed off of the instructor's energy.

Data and Analysis

In Fall 2024, Spring 2025, Fall 2025, and Spring 2026 semesters, students took a Flag Maker pre-quiz that had 5 multiple-choice questions and a post-quiz with these same questions as well as a survey. The results of these quizzes and surveys are still being analyzed and will be included in this chapter soon.

2.4 New Plugged-in Activities

2.4.1 Plugged-In Earthquake Tracker Assignment

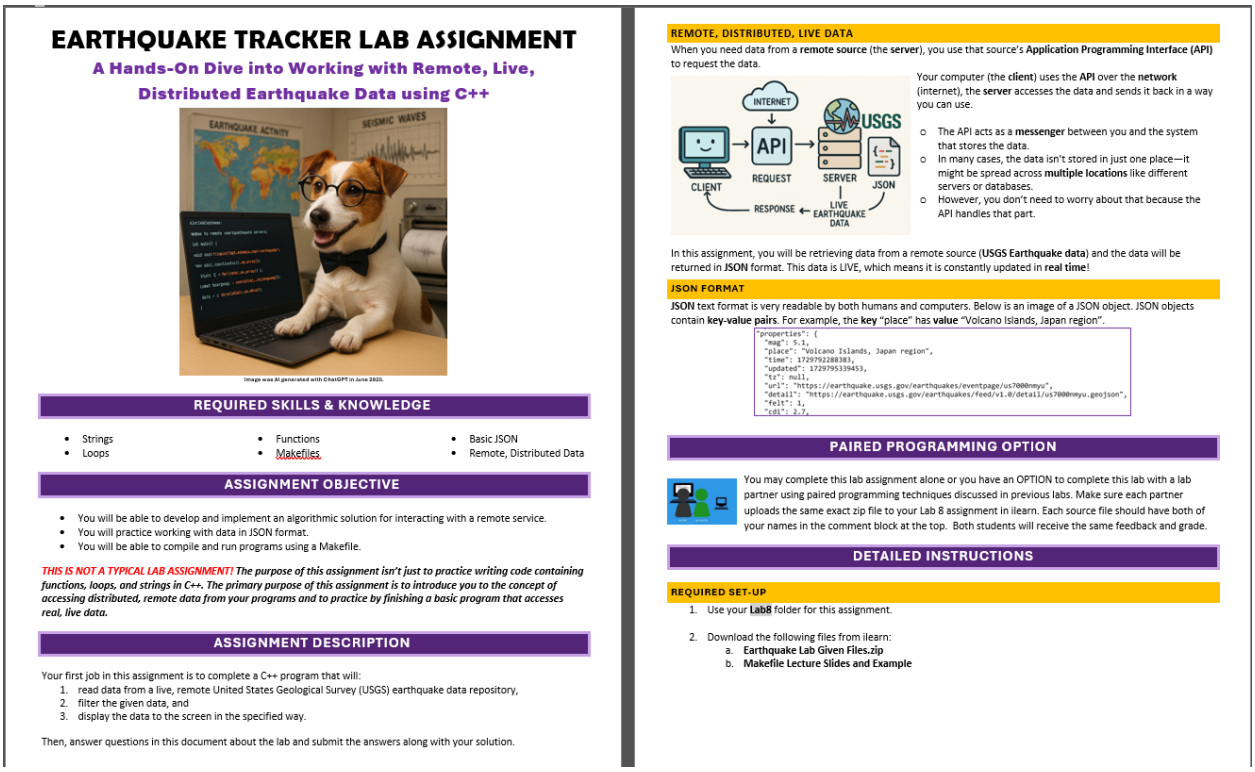
Problem Description, Prerequisites, and Learning Outcomes

Details on this assignment are in Section 1.4.4. At TNTECH, we cover these topics in week 7 & 8 (out of 16) and then students typically are assigned this programming assignment during week 8 of the semester. They have three weeks to complete the assignment.

The **learning outcome** for this activity is that students will be able to develop and implement algorithmic solutions for interacting with a remote service.

Implementation Details

Students begin with a provided starter file, `Prog3_given.cpp`, which they rename to `Prog3.cpp`. They also receive supporting folders and files, including cURL files, a



EARTHQUAKE TRACKER LAB ASSIGNMENT

A Hands-On Dive into Working with Remote, Live, Distributed Earthquake Data using C++

REQUIRED SKILLS & KNOWLEDGE

- Strings
- Loops
- Functions
- Makefiles
- Basic JSON
- Remote, Distributed Data

ASSIGNMENT OBJECTIVE

- You will be able to develop and implement an algorithmic solution for interacting with a remote service.
- You will practice working with data in JSON format.
- You will be able to compile and run programs using a Makefile.

THIS IS NOT A TYPICAL LAB ASSIGNMENT! ***The purpose of this assignment isn't just to practice writing code containing functions, loops, and strings in C++. The primary purpose of this assignment is to introduce you to the concept of accessing distributed, remote data from your programs and to practice by finishing a basic program that accesses real, live data.***

ASSIGNMENT DESCRIPTION

Your first job in this assignment is to complete a C++ program that will:

1. read data from a live, remote United States Geological Survey (USGS) earthquake data repository,
2. filter the given data, and
3. display the data to the screen in the specified way.

Then, answer questions in this document about the lab and submit the answers along with your solution.

REMOTE, DISTRIBUTED, LIVE DATA

When you need data from a **remote source** (the **server**), you use that source's **Application Programming Interface (API)** to request the data.

Your computer (the **client**) uses the **API** over the **network** (internet), the **server** accesses the data and sends it back in a way you can use.

- The API acts as a **messenger** between you and the system that stores the data.
- In many cases, the data isn't stored in just one place—it might be spread across **multiple locations** like different servers or databases.
- However, you don't need to worry about that because the API handles that part.

In this assignment, you will be retrieving data from a remote source (**USGS Earthquake data**) and the data will be returned in **JSON** format. This data is LIVE, which means it is constantly updated in **real time**!

JSON FORMAT

JSON text format is very readable by both humans and computers. Below is an image of a JSON object. JSON objects contain **key-value pairs**. For example, the **key** "place" has **value** "Volcano Islands, Japan region".

PAIRED PROGRAMMING OPTION

You may complete this lab assignment alone or you have an OPTION to complete this lab with a lab partner using paired programming techniques discussed in previous labs. Make sure each partner uploads the same exact zip file to your Lab 8 assignment in ilearn. Each source file should have both of your names in the comment block at the top. Both students will receive the same feedback and grade.

DETAILED INSTRUCTIONS

REQUIRED SET-UP

1. Use your Lab8 folder for this assignment.
2. Download the following files from ilearn:
   a. **Earthquake Lab Given Files.zip**
   b. **Makefile Lecture Slides and Example**

Figure 2.10: TNTECH CS1 Screenshot of Earthquake Tracker Assignment

JSON library folder, Mac and Windows Makefile templates, and platform-specific setup instructions. The required setup asks students to place the correct Makefile directly in the Program 3 folder and, for Windows users, copy `libcurl-x64.dll` into the same folder.

The implementation uses several libraries beyond the standard introductory set. Students add `curl/curl.h` so the program can transfer data from a URL, `nlohmann/json.hpp`, so the program can access and read JSON data, and `sstream` so downloaded website data can be temporarily stored in a string stream. The assignment provides these libraries and explains their purpose so students can use them without needing to fully master external library development.

A key implementation requirement is that students should not modify the provided `downloadDataFromURL()` function. Instead, they should read the comment block and function body to understand its purpose, then use it as a helper function that returns the remote earthquake data. The `main()` function already contains some exception-handling code, including try and catch, but students are told not to modify that portion. This allows students to encounter real-world C++ structure while still staying focused on the parts of the code aligned with"our CS1 course" learning goals.

Compilation is also a major implementation component. Students do not compile this assignment using a simple one-line g++ command because the program depends on external files and library paths. Instead, they must update the provided Makefile with their local file paths, then use commands such as make, make run, and make clean. The rubric specifically evaluates whether the program compiles with no syntax errors or warnings using -Wall, runs correctly with the Makefile, and has a functional Makefile setup.

Students must submit `Prog3.cpp`, a `_documentation.docx` file, and proof that they completed the program report. The documentation file requires screenshots, timestamp conversion, short definitions of key distributed-data terms, and a real-world example of remote data use. The rubric gives 15 points for this documentation file and 5 points for program report proof, making reflection and explanation part of the overall assignment grade.

Instructional Material (Lectures, Assignments)

- Lecture Materials on Functions, Distributed Computing, JSON, cURL, Makefiles: `https://github.com/CDER-Center/CS1-CS2_Exemplar_Ebook/tree/main/CS1/chapter2-TTU/Plugged_Activities/Earthquake%20Tracker%20Assignment`
- Assignment Document and Files Given to Students: `https://github.com/CDER-Center/CS1-CS2_Exemplar_Ebook/tree/main/CS1/chapter2-TTU/Plugged_Activities/Earthquake%20Tracker%20Assignment`

Data and Analysis

We had students fill out a program report after each programming assignment and afterwards, they would upload a screenshot of the confirmation page along with their submission, which was part of their grade. In this program report, we asked students to estimate how long they spent on the program and when they started on the program. Figure 2.11 shows how much time students thought they spent on the assignment reported from students in Fall 2024 semester through Spring 2026 semester. Figure 2.12 shows how much time students felt they spent on the assignment just in Spring 2026 disaggregated by student's prior experience in programming before taking CS1.

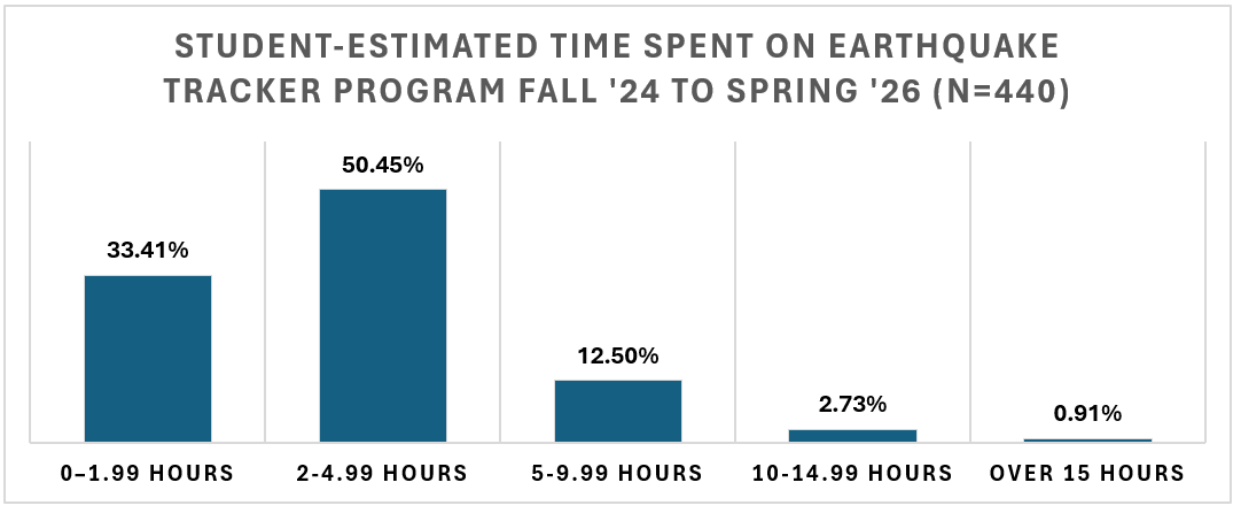


Figure 2.11: TNTECH CS1 Time to Complete Earthquake Assignment (All Semesters)

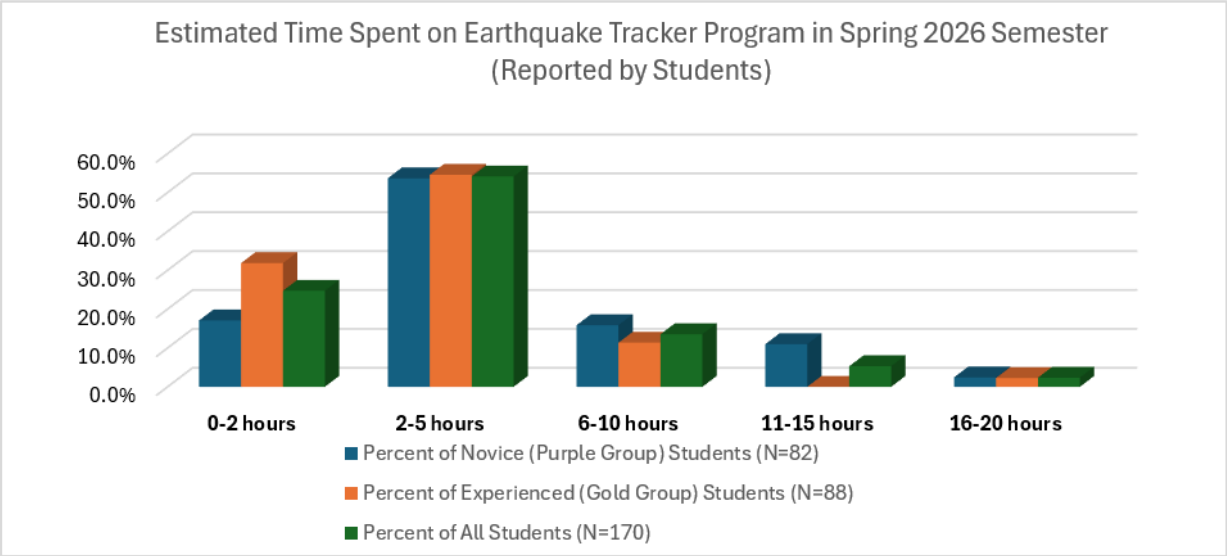


Figure 2.12: TNTECH CS1 Estimated Time to Complete Earthquake Tracker Assignment

We also asked students what they learned from the assignment. The strongest learning outcome was that students saw how a C++ program can interact with remote, live, real-world data. Many responses specifically mentioned pulling data from a website, accessing a server, using an API, or retrieving live information. This suggests that the assignment successfully exposed students to a more authentic use of programming beyond isolated console input/output. A second major learning area was Makefiles and compilation. This is interesting because Makefiles were probably not the conceptual centerpiece of the assignment, but they became highly salient to students. Many students said they learned how to use or run a Makefile, while others expressed frustration with them. This suggests that the build/setup process was a major part of the student experience. A third major theme was JSON. Students frequently mentioned learning how JSON works, how it is structured, or how to extract information from a JSON file. Several students connected JSON with APIs and live data, which suggests that they were beginning to understand the larger workflow: retrieve remote data, parse the JSON, and display selected information.

The program report also asked students what the most challenging aspect was of the assignment. The most frequent difficulties involved using the Makefile, compiling and running the program from the terminal, configuring file paths, and resolving external library issues such as cURL headers or missing `.dll` files. Many students experienced the assignment as a lesson in getting the build environment to work, which wasn't the intention.

Figure 2.13 shows the results to three statements that we asked students to rate their level of agreement given a 7-point Likert scale.

Experience and Teaching Tips

Although many students found value in the assignment and recognized the relevance of APIs, JSON, remote data, and live real-world information, we feel that the assignment would be more enjoyable if there wasn't such a heavy lift for learning compilation, depen-

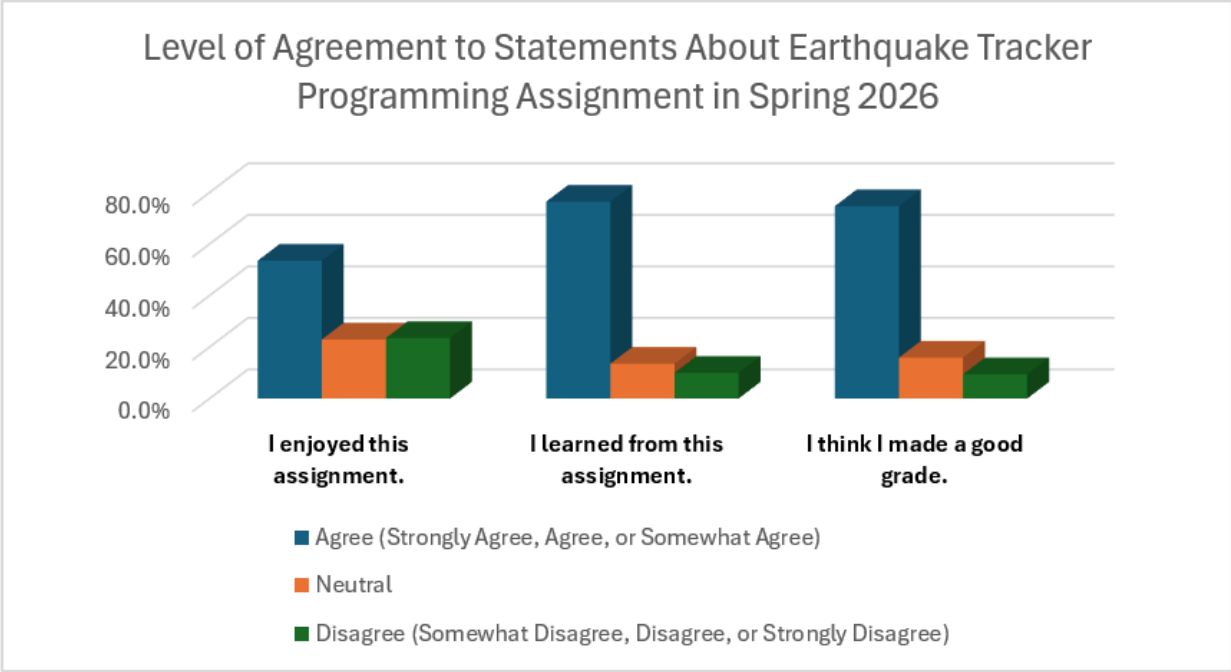


Figure 2.13: TNTECH CS1 Level of Agreement Earthquake Tracker Assignment

dencies, and environment setup. For future iterations, we are going to consider adding short "tooling checkpoints" in lecture or lab before the full assignment, where students only verify that cURL, JSON support, and the Makefile work on their computer. That could reduce frustration and allow the main assignment to focus more clearly on the intended learning outcomes.

One teaching tip is to make sure to include requiring the documentation questions because they are pedagogically valuable. They give students a chance to explain the computing concepts in plain language and connect the assignment to public-service systems beyond earthquakes, such as weather forecasting, emergency alerts, traffic navigation, disease tracking, or flight monitoring. These questions can help students recognize that APIs and distributed data are not abstract ideas; they are part of everyday systems people rely on.

2.4.2 Plugged-In OpenMP Data Parallelism Assignment

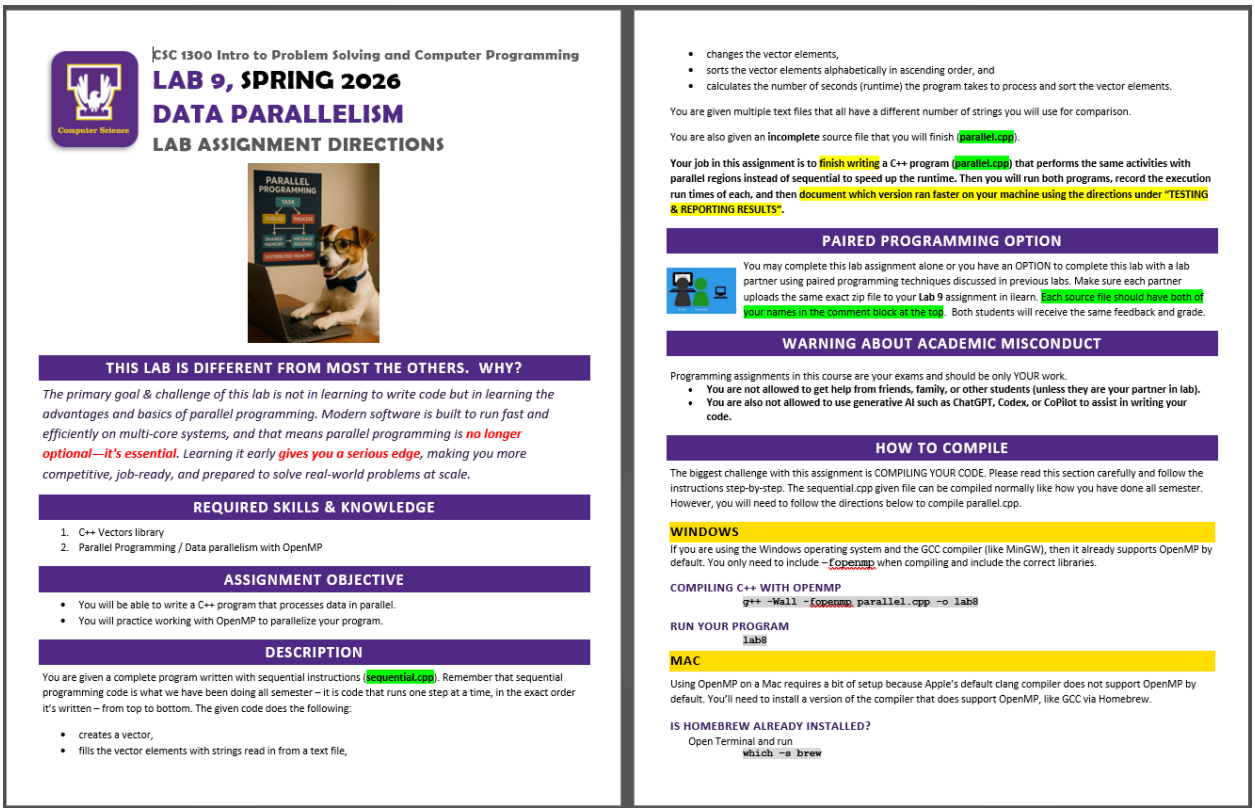
Problem Description, Prerequisites, and Learning Outcomes

This lab introduces students to data parallelism through a hands-on comparison between a sequential C++ program and an OpenMP-based parallel version. Students are given a completed sequential program, `sequential.cpp`, that reads strings from a text file into a vector, modifies each string by prepending its character count, sorts the vector alphabetically using bubble sort, and records the program's runtime. Students are also given an incomplete `parallel.cpp` file, which they must modify so that the data-processing portion of the program uses OpenMP parallelism. After implementing the parallel version, students run both programs on input files of increasing size and compare the execution times.

The purpose of the lab is not primarily to teach a new programming structure from scratch, but to help students observe how parallelism affects runtime, especially as data size increases. The lab emphasizes that parallel programming is increasingly important because modern computers rely on multi-core processors, and efficient software often needs to take advantage of that hardware. Students also see that parallelism is not automatically faster in all cases: for small input sizes, the overhead of creating and managing threads may make the parallel version slower, while larger input sizes may show a clearer benefit.

The **prerequisites** for this lab assignment include being comfortable with C++ vectors, loops, functions, file input, strings, and basic sorting logic. They also need introductory exposure to OpenMP concepts, including threads, parallel regions, and the `#pragma omp parallel` for directive. Because compiling OpenMP programs requires different commands than compiling standard C++ programs, students also need basic command-line compiling experience and some familiarity with their operating system's compiler setup.

Overall, the **learning outcome** for this activity is that students will have the ability to implement algorithmic solutions that are representative of data parallelism. Students should be able to explain the difference between sequential and parallel execution, use OpenMP to parallelize a loop, compile a C++ program with OpenMP support, collect runtime data across multiple input sizes, and interpret when parallelism improves performance and when it does not. They should also be able to connect their results to hardware factors such as the number of cores on their own machine.



CSC 1300 Intro to Problem Solving and Computer Programming

LAB 9, SPRING 2026

DATA PARALLELISM

LAB ASSIGNMENT DIRECTIONS

THIS LAB IS DIFFERENT FROM MOST THE OTHERS. WHY?

*The primary goal & challenge of this lab is not in learning to write code but in learning the advantages and basics of parallel programming. Modern software is built to run fast and efficiently on multi-core systems, and that means parallel programming is **no longer optional—it's essential**. Learning it early **gives you a serious edge**, making you more competitive, job-ready, and prepared to solve real-world problems at scale.*

REQUIRED SKILLS & KNOWLEDGE

1. C++ Vectors library
2. Parallel Programming / Data parallelism with OpenMP

ASSIGNMENT OBJECTIVE

- You will be able to write a C++ program that processes data in parallel.
- You will practice working with OpenMP to parallelize your program.

DESCRIPTION

You are given a complete program written with sequential instructions (sequential.cpp). Remember that sequential programming code is what we have been doing all semester – it is code that runs one step at a time, in the exact order it's written – from top to bottom. The given code does the following:

- creates a vector,
- fills the vector elements with strings read in from a text file,
- changes the vector elements,
- sorts the vector elements alphabetically in ascending order, and
- calculates the number of seconds (runtime) the program takes to process and sort the vector elements.

You are given multiple text files that all have a different number of strings you will use for comparison.

You are also given an **incomplete** source file that you will finish (parallel.cpp).

Your job in this assignment is to finish writing a C++ program (parallel.cpp) that performs the same activities with parallel regions instead of sequential to speed up the runtime. Then you will run both programs, record the execution run times of each, and then document which version ran faster on your machine using the directions under "TESTING & REPORTING RESULTS".

PAIRED PROGRAMMING OPTION

You may complete this lab assignment alone or you have an OPTION to complete this lab with a lab partner using paired programming techniques discussed in previous labs. Make sure each partner uploads the same exact zip file to your **Lab 9** assignment in ilearn. Each source file should have both of your names in the comment block at the top. Both students will receive the same feedback and grade.

WARNING ABOUT ACADEMIC MISCONDUCT

Programming assignments in this course are your exams and should be only YOUR work.

- **You are not allowed to get help from friends, family, or other students (unless they are your partner in lab).**
- **You are also not allowed to use generative AI such as ChatGPT, Codex, or CoPilot to assist in writing your code.**

HOW TO COMPILE

The biggest challenge with this assignment is COMPILING YOUR CODE. Please read this section carefully and follow the instructions step-by-step. The sequential.cpp given file can be compiled normally like how you have done all semester. However, you will need to follow the directions below to compile parallel.cpp.

WINDOWS

If you are using the Windows operating system and the GCC compiler (like MinGW), then it already supports OpenMP by default. You only need to include –fopenmp when compiling and include the correct libraries.

COMPILING C++ WITH OPENMP

```
g++ -Wall -fopenmp parallel.cpp -o lab8
```

RUN YOUR PROGRAM

```
lab8
```

MAC

Using OpenMP on a Mac requires a bit of setup because Apple's default clang compiler does not support OpenMP by default. You'll need to install a version of the compiler that does support OpenMP, like GCC via Homebrew.

IS HOMEBREW ALREADY INSTALLED?

Open Terminal and run

```
which -a brew
```

Figure 2.14: TNTECH CS1 Screenshot of Plugged-In OpenMP Data Parallelism Assignment

Algorithms

The assignment centers on two related algorithms: a sequential data-processing algorithm and a parallelized version of that same algorithm.

In the sequential version, the program reads a list of strings from a text file and stores them in a vector. It then processes each string one at a time. For every string in the vector, the program counts the number of characters, converts that number to a string, and prepends it to the original word using the format `count_word`. For example, apricot becomes `7_apricot`, and cat becomes `3_cat`. After every string has been modified, the program calls a bubble sort function to sort the vector in ascending alphabetical order. The program records the time required for the processing and sorting steps.

The parallel version preserves the same overall logic and output goal but changes how the string-processing loop is executed. Instead of processing each vector element one at a time in a strictly sequential loop, the program uses OpenMP to divide iterations of the loop among multiple threads. Because each string can be modified independently, this portion of the program is a good example of data parallelism: the same operation is applied to many independent elements in a collection. Students explicitly set the number of threads, then use an OpenMP directive before a traditional C++ for loop so that multiple elements can be processed at the same time.

The sorting step still uses bubble sort, which is intentionally simple but inefficient for large datasets. This helps make the cost of processing and sorting visible in the runtime results. Bubble sort repeatedly compares adjacent elements and swaps them when they are out of order. Although it is easy for beginning students to understand, it grows very slowly as the number of elements increases, making it useful for demonstrating how runtime changes as input size grows.

The lab also introduces an important performance concept: parallel speedup depends on both the amount of work being parallelized and the overhead required to manage threads. For small files, such as the file with 374 strings, the parallel version may be slower because the overhead costs are larger than the benefit of dividing the work. For larger files, students are more likely to see the parallel version improve runtime.

Implementation Details

We prepare students for this assignment during a 55-minute lecture during week 9 after students have already been introduced to loops (during week 6) and arrays (during weeks 7 & 8). During this lecture, task parallelism and data parallelism is introduced and explained. Then, OpenMP is introduced and worked examples using OpenMP is provided and demonstrated. We first show a sequential example program running on a small file, then explain where the string-processing loop occurs. Next, we show how the same loop can be rewritten as a traditional index-based for loop and prepared for OpenMP.

Then, during the 75-minute lab class, this lab assignment is given to students. The assignment itself gives students several files of increasing size: words_374.txt, words_9330.txt, words_18660.txt, words_31100.txt, and words_466493.txt. These files allow students to observe how runtime changes as input size increases. Students were instructed to test correctness first with the smallest file, where printing the vector is still manageable, and then

disable vector printing before collecting runtime data on the larger files.

Instructional Material (Lectures, Assignments)

- Lecture Materials on Arrays, Vectors, Parallelism, and OpenMP: `https://github.com/CDER-Center/CS1-CS2_Exemplar_Ebook/tree/main/CS1/chapter2-TTU/Plugged_Activities/OpenMP%20Data%20Parallelism%20Assignment`
- Assignment Document and Files Given to Students during the Lab: `https://github.com/CDER-Center/CS1-CS2_Exemplar_Ebook/tree/main/CS1/chapter2-TTU/Plugged_Activities/OpenMP%20Data%20Parallelism%20Assignment`

How-To Guide

Students need a clear guide for getting OpenMP set up in their particular computing environment and a guide for compiling OpenMP programs. The assignment contains separate Windows and Mac instructions because the setup differs substantially. On Windows with GCC or MinGW, students compile with the -fopenmp flag: `g++ -Wall -fopenmp parallel.cpp -o lab9` flag. On Mac, students may need to install Homebrew and then install GCC because Apple's default clang compiler does not support OpenMP by default. After installing GCC through Homebrew, students need to compile using a versioned compiler command and -fopenmp flag: `g++-14 -fopenmp -Wall parallel.cpp -o lab9`. (The exact version number may vary depending on the GCC version installed.)

Students also receive a short guide for checking the number of cores on their computer. This hardware information is needed to help teach students that runtime results are machine-dependent. Another useful guide provided in the assignment is a testing workflow. Students are instructed to first compile and run `sequential.cpp` to understand the baseline behavior. Then they are instructed to compile and run `parallel.cpp` with the smallest file, confirm that the output is correct, and only then move to larger files. Students are reminded to comment out or disable the `printVector()` function call before collecting timing results because printing a large vector will distort runtime measurements. They are also reminded to close unnecessary programs and avoid running heavy background tasks while timing their code.

Students are also given a simple results template along with the other given files. This template helps students focus on collecting and interpreting their data rather than guessing how to format the report.

Experience and Teaching Tips

This lab works best when students understand from the beginning that the goal is not simply to "make the code faster". The deeper goal is to investigate when and why parallelism helps. Some students may expect the parallel version to always be faster, so it is useful to explicitly discuss overhead before they begin. Thread creation, scheduling, and coordination

all have costs. For small input sizes, those costs may outweigh the benefit of parallel execution. This makes the smaller input files pedagogically valuable because they help students confront a common misconception about parallel programming.

Compiling is likely to be the biggest practical obstacle, especially for Mac users. It is helpful to devote class or lab time to confirming that students can compile a basic OpenMP program before they work on the full assignment. They can use the provided program examples to test their computer. Students who cannot get OpenMP working may become stuck even if they understand the programming concept. A short "compiler check" activity before the lab can prevent many issues.

Students may also struggle with the difference between `for_each` and a traditional `for` loop. Since the starter code uses `for_each`, students need to understand why they are being asked to rewrite that portion. The clearest explanation is that OpenMP loop parallelization works naturally with standard index-based loops where the compiler can divide loop iterations among threads.

Another important teaching point is variable scope inside the parallel loop. Students should define the character-count variable inside the loop so that each thread has its own local copy. This helps avoid accidental sharing of a variable across threads and gives the instructor an opportunity to introduce the broader idea of shared versus private data in parallel programming.

It is also useful to frame the thread-count experiment as a discovery activity. When students change the number of threads to 2 and then to 500, they can see that more threads do not always mean better performance. Too few threads may not use the hardware effectively, while too many threads can create significant overhead. This gives students a more realistic understanding of performance tuning.

Finally, students should be reassured that their runtimes will not exactly match the sample output. Runtime depends on hardware, number of cores, operating system, compiler, background processes, and current system load. The most important outcome is not matching the instructor's times exactly, but collecting consistent results and explaining them thoughtfully.

Data and Analysis

As part of the lab assignment grade, students must fill out a lab report and upload the confirmation screenshot along with their submission to prove that they submitted the report. Students always have 6 days from the date of their lab class to submit assignments. Figure 2.15 shows student responses to asking "How many total hours did you spend on this lab assignment?"

Figure 2.16 shows the common themes of responses from students when they were asked the open-ended question "What is something that you learned from this assignment?" The strongest theme was students' increased understanding of parallel computing. Nearly half of the respondents explicitly mentioned parallel programming, parallel processing, parallelism, or the difference between parallel and sequential execution. Responses ranged from basic recognition to more developed explanations of how work can be divided across multiple processing resources.

A particularly important learning outcome involved performance and efficiency. Many

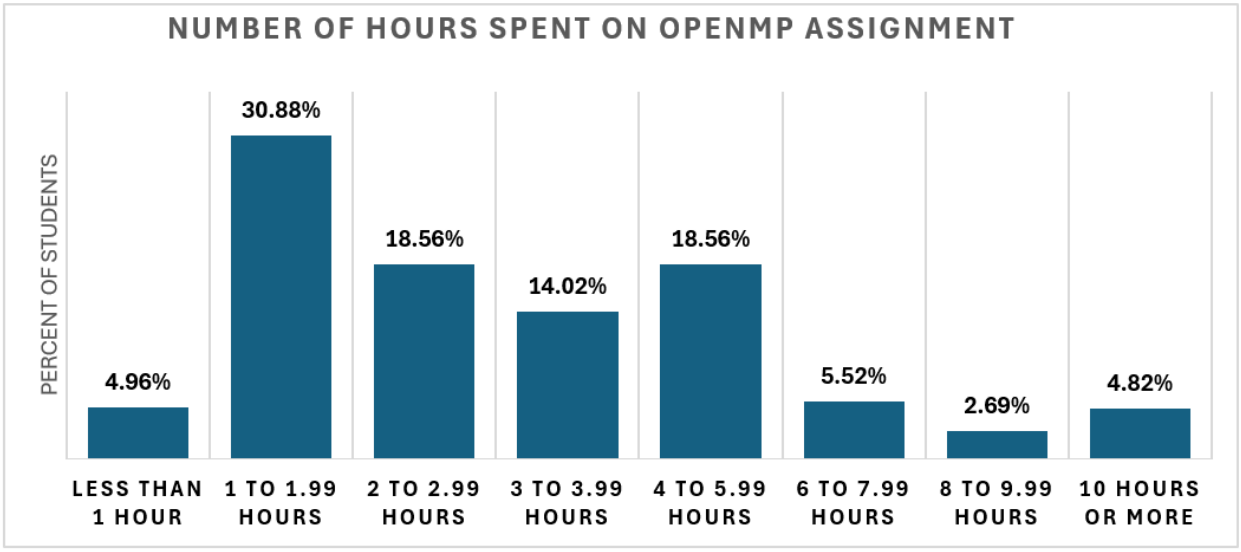


Figure 2.15: TNTECH CS1 Hours Spent on Plugged-In OpenMP Lab Assignment

students recognized that parallel execution can reduce runtime, but they also learned that parallelization is not automatically faster. Students observed that performance depends on the task, the amount of data, the number of threads, and the computer's available cores. Several responses specifically noted that adding more threads does not always improve speed and can sometimes make a program slower. This suggests that students developed a more nuanced understanding than simply equating parallelism with improved performance.

Students also reported substantial learning related to arrays and data organization. These responses included using one- and two-dimensional arrays, iterating through arrays with loops, passing arrays to functions, and organizing information within a program. This indicates that the assignment reinforced foundational programming concepts while introducing parallel-computing ideas.

Below is a list of common challenges reported by students when they were asked the open-ended question "What was the most challenging aspect of the assignment? Is there anything you still have questions about?"

- Difficulty implementing parallel code or interpreting unexpected performance results
- Challenges creating, accessing, passing, or correctly indexing arrays
- Locating errors, correcting mistakes, and making output match expected results
- Difficulties defining functions, passing data, using references, and organizing files
- Problems compiling, configuring libraries, using run.sh, or working across operating systems
- Problems calculating values, processing files, and formatting or printing results
- Challenges writing loops, nested loops, parallel loops, and conditional statements

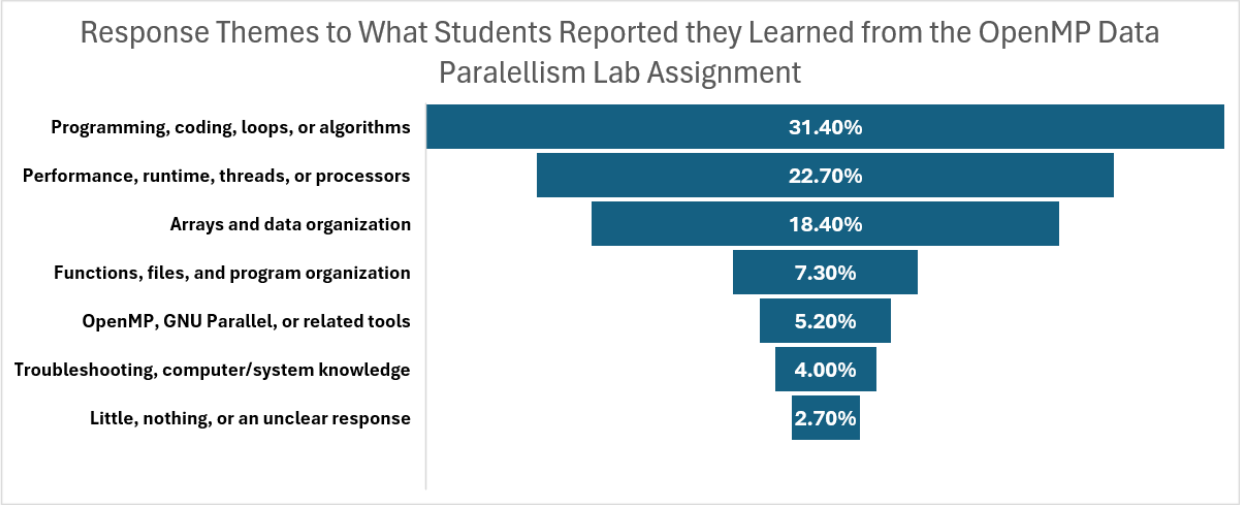


Figure 2.16: TNTECH CS1 What Students Learned from Plugged-In OpenMP Data Parallelism Assignment

Students were also asked their level of agreement to two statements where students were provided a 7-point Likert scale. One statement was "I enjoyed this assignment" and the other was "I learned from this assignment." Figure 2.17 shows the results from students answering these questions.

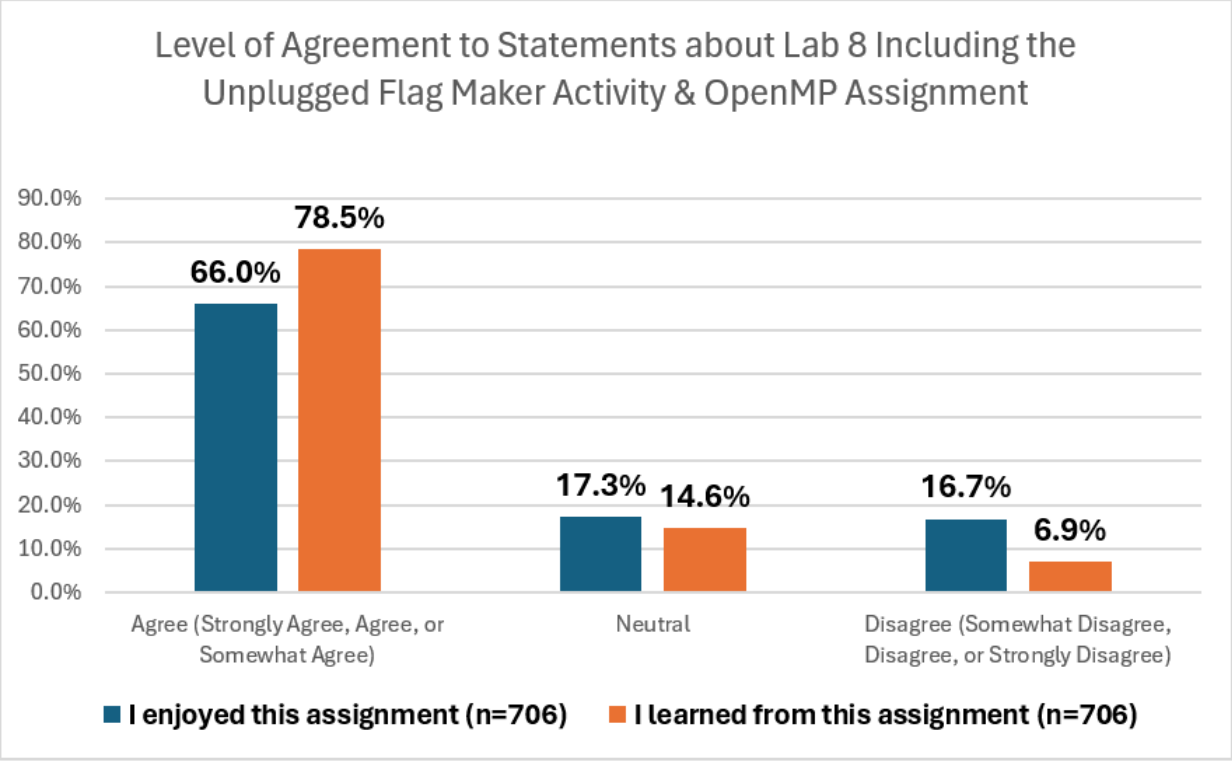


Figure 2.17: TNTECH CS1 Level of Agreement OpenMP Data Parallelism Lab Assignment

2.5 Course-wide Pre and Post Course Surveys

To determine if our new and modified modules and assignments were effective at teaching basic PDC concepts while not reducing understanding of core, fundamental CS1 topics, we administered pre- and post-course surveys. Details about these surveys can be found in Section 1.5. To download the actual surveys performed in CS1 and CS2 at TNTECH, go to `https://github.com/CDER-Center/CS1-CS2_Exemplar_Ebook/tree/main/CS1/chapter2-TTU/Evaluation_Data_and_Analysis/Survey_Instruments`. Table 11.7 shows how many students were enrolled in the CS1 course from Fall 2024 to Spring 2026. The table also shows how many consenting students filled out the Pre-Course survey and Post-Course survey each semester. These students' results are those reported in Section 2.5.1.

Table 2.9: TNTECH Sample Size for Pre-Course and Post-Course Surveys

Population	Spr 2024	Fall 2024	Spr 2025	Fall 2025	Spr 2026	Total
No. Students Enrolled in CS1 Course	180	253	191	255	194	1256
No. Consented Students Pre-Course Survey	130	223	236	169	173	1090 (86.8%)
No. Consented Matched Pre-Post Surveys	127	166	123	196	140	879 (70%)

2.5.1 Results of Pre-Course and Post-Course Surveys

Spring 2024 CS1 Baseline Results

Table 2.10 summarizes the pre- and post-course survey results from the Spring 2024 benchmark semester, during which no formal PDC intervention was implemented. The **Pre** and **Post** columns present the mean student ratings collected at the beginning and end of the course, respectively. Responses were measured using a seven-point scale ranging from 1, indicating no experience at all, to 7, indicating an extremely high level of experience. Therefore, higher mean values indicate that students perceived themselves as having more experience with the concept, while lower values indicate little or no perceived experience. The **Diff** column represents the post-course mean minus the pre-course mean and can theoretically range from -6 to $+6$. A positive value indicates that students' average reported experience increased during the course, a value near zero indicates little change, and a negative value indicates that the average rating decreased. Larger positive differences represent greater pre-to-post growth. The **W** column reports the Wilcoxon signed-rank test statistic, which is calculated from the ranks of the differences between matched pre- and post-course responses. The value of W is interpreted together with the sample size and p-value; therefore,

a high or low W value does not independently indicate whether the amount of improvement was large or important. The **p-value** column reports the probability of obtaining results at least as extreme as those observed if there were no systematic difference between the pre- and post-course responses. Values range from 0 to 1, with smaller values providing stronger evidence of a pre-to-post difference. In these analyses, results with a p-value below .05 were considered statistically significant. The **Sig.** column summarizes this decision as "Yes" when p $<$ .05 and "No" when $p >= .05$. The **Cliff's** δ column provides an effect-size estimate ranging from -1 to $+1$. Because Cliff's delta was calculated in the pre-to-post direction, negative values indicate that post-course ratings generally tended to be higher than pre-course ratings, while positive values indicate that pre-course ratings tended to be higher. Values close to zero indicate little separation between the pre- and post-course responses, whereas values closer to either -1 or $+1$ indicate a stronger and more consistent difference. The magnitude of the value, rather than its sign, is used to determine the size of the effect. The **Interp.** column categorizes that magnitude as negligible, small, medium, or large.

Table 2.10 shows that students reported statistically significant increases in their programming experience and in every traditional computing concept measured, with large effect sizes for data types, mathematical expressions, branching, loops, functions, reference variables, arrays, pointers, structures, and classes and objects. Among the PDC concepts, students reported a statistically significant but relatively small increase in experience with parallel processing, from 1.69 to 2.45. In contrast, experience with distributed computing remained essentially unchanged, while reported experience with event handling decreased from 2.65 to 1.74. Table 3.10

Fall 2024 through Spring 2026 CS1 Intervention Results

Table 2.11 presents the combined results from the Fall 2024 through Spring 2026 intervention semesters. Students again demonstrated statistically significant growth across all traditional CS1 concepts, with large effect sizes, indicating that incorporating PDC material did not interfere with their development of foundational programming knowledge. In addition, students reported significant gains in all three PDC areas: parallel processing increased from 1.83 to 3.77, distributed computing increased from 1.80 to 3.40, and event handling increased from 2.71 to 3.47. Parallel processing and distributed computing produced large effect sizes, while event handling produced a small effect size.

Conclusion of Results

The overall survey results suggest that students made significant pre-to-post gains across both traditional CS1 topics and PDC-related concepts. After the PDC intervention, students continued to report strong growth in core programming knowledge, indicating that the added PDC material did not undermine foundational CS1 learning. For PDC topics, students showed statistically significant growth in parallel processing, distributed computing, and event handling. However, compared with the Spring 2024 benchmark, the size of the gains was similar rather than substantially larger. Parallel processing and distributed computing showed slight increases in mean gain after the intervention, while event handling showed a smaller gain, consistent with its limited instructional emphasis. Overall, the data supports

Table 2.10: TNTECH CS1 Baseline Results: Spring 2024

Item	Pre	Post	Diff	W	p-value	Sig.	Cliff's δ	Interp.
Computing Background								
Computer Experience	5.54	5.66	0.12	2094.0	0.167	No	-0.04	negligible
Prog. Experience	3.44	4.58	1.14	463.0	0.000	Yes	-0.38	medium
Prog. Experience Compared to Classmates	3.23	4.39	1.16	735.0	0.000	Yes	-0.40	medium
Computing Concepts								
Data Types	3.57	6.52	2.95	0.0	0.000	Yes	-0.83	large
Math Expressions	3.57	6.41	2.84	0.0	0.000	Yes	-0.77	large
Branching	3.41	6.42	3.01	0.0	0.000	Yes	-0.81	large
Loops	4.07	6.43	2.36	23.0	0.000	Yes	-0.75	large
Functions	3.50	6.02	2.52	30.0	0.000	Yes	-0.74	large
Reference Variables	3.07	5.45	2.38	38.5	0.000	Yes	-0.66	large
Arrays	2.45	5.61	3.16	39.0	0.000	Yes	-0.81	large
Pointers	1.81	4.67	2.86	74.0	0.000	Yes	-0.77	large
C++ Structures	1.73	5.80	4.07	22.0	0.000	Yes	-0.90	large
Classes & Objects	2.43	4.84	2.41	158.0	0.000	Yes	-0.68	large
PDC Concepts								
Parallel Processing	1.69	2.45	0.76	534.0	0.000	Yes	-0.28	small
Distributed Computing	1.61	1.64	0.02	551.5	0.891	No	-0.01	negligible
Event Handling	2.65	1.74	-0.91	641.5	0.000	Yes	0.32	small

the conclusion that PDC topics can be introduced into CS1 in a way that is feasible and non-disruptive, but larger gains in students' perceived PDC experience may require more explicit engagement with these concepts.

2.6 Conclusion

While most PDC efforts have focused on programming strategies for upper-level and elective courses, we believe there is value and opportunity in introducing these concepts much earlier in the computing curriculum. Our studies have demonstrated that our approach has statistically significant and substantial gains in both learning and appreciation for PDC topics. We hope that other educators can adopt or adapt our modules and activities and replicate our results and success.

SS: Rewrite to include Neena's points on:

- Lessons Learned
  - During the benchmark semester, we didn't ask students in the post-course survey what their experience was with the different programming languages. We also

didn't ask any of the computing attitudes questions in either the . We realized that it would be

  - While students perceived that their understanding of PDC topics were significantly higher in the post-course survey compared to the pre-course survey, their perception of importance of of PDC was not significantly higher. The pre-course survey revealed that students come into the course already thinking that PDC is important in computing (pre-course mean of 5.51) and so there isn't room for growth perception of importance.
  -

- Teaching Tips for Instructors
- Inferences and Limitations
- Future Work

Table 2.11: TNTECH CS1 Intervention Results: Fall 2024, Spring 2025, Fall 2025, and Spring 2026 Combined

Item	Pre	Post	Diff	W	p-value	Sig.	Cliff's δ	Interp.
Computing Background								
Computer Experience	5.54	5.65	0.11	29995.0	0.019	Yes	-0.03	negligible
Prog. Experience	3.41	4.66	1.26	6507.0	0.000	Yes	-0.44	medium
Prog. Experience Compared to Classmates	3.17	4.35	1.19	10330.5	0.000	Yes	-0.43	medium
Programming Languages								
C	1.30	1.70	0.40	325.0	0.000	Yes	-0.17	small
C++	1.70	4.66	2.96	80.5	0.000	Yes	-0.87	large
C#	1.28	1.52	0.23	247.0	0.000	Yes	-0.12	negligible
Java	1.78	1.84	0.06	1078.5	0.419	No	0.01	negligible
Javascript	1.86	1.86	-0.00	1546.5	0.866	No	0.01	negligible
PHP	1.11	1.12	0.01	68.0	1.000	No	-0.01	negligible
Python	2.72	2.78	0.06	3487.5	0.573	No	-0.01	negligible
Scratch	2.06	1.96	-0.10	1139.0	0.085	No	0.05	negligible
Visual Basic	1.19	1.30	0.11	185.0	0.133	No	-0.03	negligible
Other	1.90	1.89	-0.01	348.0	0.742	No	0.03	negligible
Computing Concepts								
Data Types	2.86	5.76	2.90	784.5	0.000	Yes	-0.73	large
Math Expressions	3.15	5.68	2.53	1334.5	0.000	Yes	-0.67	large
Branching	3.09	5.68	2.59	1132.5	0.000	Yes	-0.71	large
Loops	3.23	5.69	2.46	1098.0	0.000	Yes	-0.70	large
Functions	3.18	5.40	2.22	2356.5	0.000	Yes	-0.64	large
Reference Variables	2.83	4.91	2.07	4950.0	0.000	Yes	-0.62	large
Arrays	2.39	5.06	2.67	1511.0	0.000	Yes	-0.73	large
Pointers	1.63	4.28	2.66	787.5	0.000	Yes	-0.83	large
C++ Structures	1.43	4.68	3.25	339.0	0.000	Yes	-0.89	large
Classes & Objects	2.18	4.08	1.90	5188.5	0.000	Yes	-0.62	large
PDC Concepts								
Parallel Processing	1.83	3.77	1.94	8082.0	0.000	Yes	-0.68	large
Distributed Comp.	1.80	3.40	1.60	11266.0	0.000	Yes	-0.58	large
Event Handling	2.71	3.47	0.76	33627.0	0.000	Yes	-0.25	small
Computing Attitudes								
Interest in Computing	5.72	5.69	-0.03	26850.0	0.571	No	0.00	negligible
Interest in PDC	5.19	4.99	-0.20	35928.0	0.001	Yes	0.07	negligible
Importance of PDC	5.51	5.84	0.33	28566.0	0.000	Yes	-0.17	small
Understanding of Computing	4.22	5.28	1.05	13076.5	0.000	Yes	-0.37	medium
Understanding of PDC	3.13	4.75	1.63	9785.0	0.000	Yes	-0.56	large

Bibliography

[1] OpenMP Architecture Review Board. Openmp application programming interface (api) specification, version 6.0. Technical report, OpenMP Architecture Review Board, November 2024. Online. Available: https://www.openmp.org/specifications/.

[2] April R. Crockett. Self-selected experience-based grouping in CS1: Examining student success and persistence in CS major. pages 1742–1742. ACM, 2 2026.

[3] April R. Crockett and Gerald C. Gannod. Beyond CS1: Longitudinal effects of student-selected experience-based grouping on success and persistence. In 2026 IEEE Frontiers in Education Conference (FIE), pages 1–9. Institute of Electrical and Electronics Engineers (IEEE), 2 2026.

[4] April R. Crockett, Gerald C. Gannod, and Moumita Kamal. Improving student success and retention in CS1 through self-selection into experience-based groups. In 2023 IEEE Frontiers in Education Conference (FIE), pages 1–9. IEEE, 10 2023.

[5] April R. Crockett, Gerald C. Gannod, and Neena Thota. Enhancing student success in CS1: A self-selected experience-based grouping approach. In 2025 IEEE Frontiers in Education Conference (FIE), pages 1–9. Institute of Electrical and Electronics Engineers (IEEE), 1 2025.

[6] Leonardo Dagum and Ramesh Menon. Openmp: an industry standard api for shared-memory programming. IEEE computational science and engineering, 5(1):46–55, 1998.

[7] Amruth N Kumar, Rajendra K Raj, Sherif G Aly, Monica D Anderson, Brett A Becker, Richard L Blumenthal, Eric Eaton, Susan L Epstein, Michael Goldweber, Pankaj Jalote, et al. Computer science curricula 2023, 2024.

[8] Chase Lambert. Makefile Tutorial by Example. Technical report, Makefile Tutorial, July 2026. Online; available at https://makefiletutorial.com/.

[9] Niels Lohmann. JSON for Modern C++. Technical report, nlohmann/json, July 2025. Online. Available: https://json.nlohmann.me/.

[10] Sushil K Prasad, Almadena Yu Chtchelkanova, Sajal K Das, Frank Dehne, Mohamed G Gouda, Anshul Gupta, Joseph Jaja, Krishna Kant, Anita La Salle, Richard LeBlanc, et al. NSF/IEEE-TCPP curriculum initiative on parallel and distributed computing: Core topics for undergraduates. In SIGCSE, volume 11, pages 617–618, 2011.

[11] Daniel Stenberg and curl contributors. curl. Technical report, The curl project, July 2026. Online; available at https://curl.se/.

[12] Frank Vahid and Roman Lysecky. Programming in C++. https://www.zybooks.com/catalog/programming-c-plus-plus/, 09 2025. Interactive online textbook.

Appendix

Github Link to Instructional Material

`https://github.com/CDER-Center/CS1-CS2_Exemplar_Ebook/tree/main/CS1/chapter2-TTU`

Detailed Pre and Post Syllabus

`https://github.com/CDER-Center/CS1-CS2_Exemplar_Ebook/tree/main/CS1/chapter2-TTU/Detailed_Pre_and_Post_Syllabus`

Organization of Material

The supporting materials for the TNTECH CS1 course package are available in the TNTECH CS1 GitHub repository. The repository is organized into the following categories:

- **Course Overview and Calendar**
  - `README.md`: Provides an overview of CSC 1300, including the institution, course level, programming language, included activities, and contact information.
  - `COURSE_CALENDAR.md`: Provides a general calendar identifying the placement of course topics and activities throughout the semester.
- **Detailed Post-Intervention Syllabus and Course Schedule** The `Detailed_Pre_and_Post_` directory contains the detailed Spring 2026 course documents:
  - CSC 1300 Spring 2026 Syllabus
  - CSC 1300 Spring 2026 Master Schedule

  These files document the organization of the course after the integration of parallel, distributed, and event-driven computing topics.
- **Plugged-In Earthquake Tracker Assignment** The Plugged-In Earthquake Tracker Assignment directory contains an assignment in which students develop an earthquake-tracking program while exploring distributed computing, remote services, and APIs.
  - Activity overview and instructions
  - Earthquake Tracker assignment directions
  - Earthquake Tracker grading rubric
  - Module 7 Functions and Distributed Computing presentation slides
  - Earthquake Program Given Files: Contains the C++ starter program, JSON-support files, and platform-specific Makefiles supplied to students.
  - CSC 1300 Makefiles presentation

  - Mac Makefile Example: Contains a driver program, function definitions, a header file, and a Makefile configured for macOS.
  - Windows Makefile Example: Contains a driver program, function definitions, a header file, and a Makefile configured for Windows.

- **Unplugged and Supplemental Instructional Activities** The `Unplugged_Activities` directory contains hands-on activities and supplemental lecture materials.
  - **Unplugged Penny Search Activity** The Unplugged Penny Search Activity introduces search strategies, task distribution, and parallel work through a hands-on exercise.
    * Activity overview and implementation information
    * Penny Search Activity presentation
    * Penny Sorting Exercise
  - **Unplugged Flag Maker Activity** The Unplugged Flag Maker Activity introduces parallel programming and data parallelism through a collaborative flag-making exercise.
    * Activity overview and implementation information
    * Flag Maker Activity presentation slides
    * Printable Flag Maker scenario grids: Contains four printable scenario grids used during the activity.
  - **Event Handling Lecture** The Event Handling Lecture directory contains materials introducing object-oriented programming, graphical user interfaces, and event handling.
    * Lecture overview
    * Module 11 Object-Oriented Programming and Event Handling presentation
  - **zyBooks Parallel Computing Materials** The zyBooks Parallel Computing directory contains supplemental materials associated with the parallel-computing content delivered through zyBooks.

- **Evaluation Data and Survey Instruments** The `Evaluation_Data_and_Analysis/Survey` directory contains informed-consent forms, course-experience surveys, activity quizzes, activity surveys, lab reports, and programming-assignment reports collected from Spring 2024 through Spring 2026.
  - **Spring 2024**
    * CS1 Spring 2024 Informed Consent Form
    * CS1 Spring 2024 Post-Course Experience Survey
  - **Fall 2024**
    * CS1 and CS2 Fall 2024 Informed Consent Form
    * CS1 Fall 2024 Pre-Course Experience Survey
    * CS1 Fall 2024 Post-Course Experience Survey

    - CS1 Fall 2024 Pre-Activity Flag Maker Quiz
    - CS1 Fall 2024 In-Lab Flag Maker Activity Survey
    - CS1 and CS2 Fall 2024 Lab Report
  - **Spring 2025**
    - CS1 and CS2 Spring 2025 Informed Consent Form
    - CS1 Spring 2025 Pre-Course Experience Survey
    - CS1 Spring 2025 Post-Course Experience Survey
    - CS1 Spring 2025 Pre-Activity Flag Maker Quiz
    - CS1 Spring 2025 In-Lab Flag Maker Activity Survey
    - CS1 and CS2 Spring 2025 Lab Report
  - **Fall 2025**
    - CS1 and CS2 Fall 2025 Informed Consent Form
    - CS1 Fall 2025 Pre-Course Experience Survey
    - CS1 Fall 2025 Post-Course Experience Survey
    - CS1 Fall 2025 Pre-Activity Flag Maker Quiz
    - CS1 Fall 2025 Post-Activity Flag Maker Quiz and Survey
    - CS1 and CS2 Fall 2025 Lab Report
    - CS1 and CS2 Fall 2025 Programming-Assignment Report
  - **Spring 2026**
    - CS1 and CS2 Spring 2026 Informed Consent Form
    - CS1 Spring 2026 Pre-Course Experience Survey
    - CS1 Spring 2026 Post-Course Experience Survey
    - CS1 Spring 2026 Pre-Activity Flag Maker Quiz
    - CS1 Spring 2026 Post-Activity Flag Maker Quiz and Survey
    - CS1 and CS2 Spring 2026 Lab Report
    - CS1 and CS2 Spring 2026 Programming-Assignment Report

- **Optional Archive of Syllabi and Course Schedules** The `Optional_Archive/Syllabi & Course Schedules` directory contains earlier syllabi and master schedules that support comparisons across implementation semesters.
  - CSC 1300 Fall 2024 Syllabus
  - CSC 1300 Fall 2024 Master Schedule
  - CSC 1300 Spring 2025 Syllabus
  - CSC 1300 Spring 2025 Master Schedule
  - CSC 1300 Fall 2025 Syllabus
  - CSC 1300 Fall 2025 Master Schedule

The instruments include informed-consent forms, pre-course and post-course experience surveys, pre-activity and post-activity quizzes, activity surveys, lab reports, and programming-assignment reports collected between Spring 2024 and Spring 2026.

- **Optional Archive** The `Optional_Archive/Syllabi & Course Schedules` directory contains earlier syllabi and master course schedules:
  - CSC 1300 Fall 2024 Syllabus
  - CSC 1300 Fall 2024 Master Schedule
  - CSC 1300 Spring 2025 Syllabus
  - CSC 1300 Spring 2025 Master Schedule
  - CSC 1300 Fall 2025 Syllabus
  - CSC 1300 Fall 2025 Master Schedule

  These archived files allow instructors and researchers to compare course organization across the baseline and intervention semesters.

Together, these materials provide the instructional resources necessary to reproduce or adapt the TNTECH implementation, as well as the course documentation and evaluation instruments needed to examine how the intervention was incorporated and assessed.

Survey and Other Anonymized Data and Analysis

Data and analysis about the added activities can be found in the individual sections about the activities:

- Unplugged Penny Search Activity Data & Analysis 2.3.1
- Unplugged Flag Maker Activity Data & Analysis 2.3.2
- Plugged-In Earthquake Tracker Assignment Data & Analysis 2.4.1
- Plugged-In OpenMP Data Parallelism Assignment Data & Analysis 2.4.2

Section 2.5.1 contains results of pre-course and post-course surveys from the Spring 2024 CS1 Baseline semester and the combined Fall 2024 through Spring 2206 CS1 PDC Intervention semesters. `https://github.com/CDER-Center/CS1-CS2_Exemplar_Ebook/tree/main/CS1/chapter2-TTU/Evaluation_Data_and_Analysis/Survey_Instruments`